

Theorem 1 (Spectral theorem). *If A is symmetric, then*

$$A = V \Lambda V^T = \sum_{i=1}^n \lambda_i v_i v_i^T$$

where Q : orthogonal, Λ : $\text{diag}(\lambda_i)$, v_i : orthonormal eigenvector.

Theorem 2. *A matrix is invertible iff it is PD.*

Linear transformation to rotate matrix counterclockwise by θ is $\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$

PDF, CDF

$$F(t) = \int_{-\infty}^t f(s) ds$$

where $F(t) = \Pr(X \leq t)$.

MVG Given $X \sim \mathcal{N}(\mu, \Sigma)$ where $\Sigma \succ 0$

$$f_X(x) = \frac{1}{(2\pi)^{d/2} \det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

and

$$M_X(t) = \mathbb{E}[e^{t^T X}] = \exp\left(t^T \mu + \frac{1}{2} t^T \Sigma t\right)$$

where for all $a \in \mathbb{R}^d$, $a^T X$ follows a univariate Gaussian distribution s.t. $\mathbb{E}[a^T X] = a^T \mathbb{E}[X] = a^T \mu$, $\text{var}(a^T X) = \mathbb{E}[(a^T X)^2] - \mathbb{E}[a^T X]^2 = a^T (\mathbb{E}[X X^T] - \mathbb{E}[X] \mathbb{E}[X^T]) a = a^T \Sigma a$. Notice that MGF of $Y = a^T X \in \mathbb{R}$ is $M_Y(s) = \mathbb{E}[e^{sY}] = \mathbb{E}[e^{s a^T X}] = \exp\left(s a^T \mu + \frac{1}{2} s^2 a^T \Sigma a\right) |_{s=1} = M_X(a)$.

$$\nabla_{\Sigma} \log(\det \Sigma) = \Sigma^{-1}$$

$$\nabla_A \text{tr}(A^{-1} B) = -A^{-1} B A^{-1} \quad A, B \in \mathbb{R}^{d \times d}$$

Level sets (i.e. contour plots) of MVG pdf are ellipses; values of x s.t. $p(x)$ (i.e. $x^T \Sigma^{-1} x$) is constant

Affine transform: if $Y = AX + b$, then

$$Y \sim \mathcal{N}(A\mu + b, A\Sigma A^T)$$

Covariance matrix: $\text{cov}(X) = \mathbb{E}[(X - \mathbb{E}[X])(X - \mathbb{E}[X])^T]$

For jointly Gaussian random variables only: uncorrelated $\iff$ independent. So if $\Sigma_{ij} = 0$, then $X_i \perp X_j$.

MoG A Gaussian mixture model $p(x)$ s.t. the probability of sampling $x \in \mathbb{R}^d$ as a weighted combination of Gaussians

$$p(x|\pi, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x|\mu_k, \Sigma_k)$$

where $\sum_{k=1}^K \pi_k = 1$ is a probability distribution over clusters; $\pi_k \geq 0$: mixture weights.

• Latent indicator variables $z_{ik} = \begin{cases} 1 & x_i \text{ assigned to cluster } k \\ 0 & \text{else} \end{cases} \quad i \in [n], k \in [K] \text{ s.t. } \sum_{k=1}^K z_{ik} = 1 \text{ where}$

z_i : one-hot vector indicating cluster assignment of x_i .

- Generative process

1. For each data point i , sample cluster indicators $z_i \sim \text{Categorical}(\pi_1, \dots, \pi_K)$.
2. Sample the observation $x_i \sim \mathcal{N}(\mu_k, \Sigma_k)$ for $z_{ik} = 1$.

$$p(x_i, z_i) = \prod_{k=1}^K (\pi_k \cdot \mathcal{N}(x_i | \mu_k, \Sigma_k))^{z_{ik}}$$

$$p(x_i) = \sum_{z_i} p(x_i, z_i) = \sum_{k=1}^K p(z_{ik} = 1) \cdot p(x_i | z_{ik} = 1) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_i | \mu_k, \Sigma_k)$$

- *Expectation-Maximization (EM) algorithm* alternates between:

1. E-step (estimate cluster membership probabilities **and assign soft labels**)

$$\gamma_{ik} = p(z_{ik} = 1 | x_i) = \frac{p(x_i, z_{ik})}{p(x_i)} = \frac{\pi_k \cdot \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

i.e. responsibility: probability that cluster k generated point i . $\sum_{k=1}^K \gamma_{ik} = 1$

Then, complete-data log-likelihood is

$$\mathbb{E}_{z|x}[\log p(x, z)] = \sum_{i=1}^n \sum_{k=1}^K p(z_{ik} = 1 | x_i) \cdot \log p(x_i, z_{ik}) = \sum_{i=1}^n \sum_{k=1}^K \gamma_{ik} \cdot \log (\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k))$$

2. M-step (update cluster parameters using those probabilities)

Fixed γ_{ik} , differentiate $\mathbb{E}_{z|x}[\log p(x, z)]$ w.r.t μ_k, Σ_k, π_k to update them. Use Lagrangian for π_k update:

$$L(\pi_1, \dots, \pi_K, \lambda) = \frac{d}{d\pi_k} \mathbb{E}_{z|x}[\log p(x, z)] + \lambda (\sum_{k=1}^K \pi_k - 1) \text{ to find } \pi_k^* = \frac{\sum_{i=1}^n \gamma_{ik}}{n}.$$

EM depends on initial parameters, not guaranteed to converge to global optimum of likelihood; unnecessary if we know responsibilities (i.e. cluster assignments).

- *K-means is MoG with zero variance.* K-means makes hard assignments (each point belongs to exactly one cluster), while MoG gives a soft fractional membership in every cluster and also models the shape/spread of each cluster, not just its center.

K-Means clustering Objective: $J(\{z_{ik}\}, \{\mu_k\}) = \sum_{k=1}^K \sum_{i=1}^n z_{ik} \|x_i - \mu_k\|_2^2$

1. Update assignments. For each point x_i , choose

$$k \in \arg \min_{j \in \{1, \dots, K\}} \|x_i - \mu_j\|_2^2$$

Let $z_{ik} = 1$, $z_{ij} = 0$ for all $j \neq k$.

2. Update centroids. For each non-empty cluster k , let

$$\mu_k = \frac{\sum_{i=1}^n z_{ik} x_i}{\sum_{i=1}^n z_{ik}}$$

Classifiers

Bayes optimal classifier minimizes expected loss over true distribution:

$$R(f) = \mathbb{E}_{(X,Y) \sim P}[L(f(X), Y)]$$

For 0-1 loss: $f^*(x) = \arg \max_C P(Y = C \mid X = x)$. **Bayes minimizes expected/population loss, NOT training error.** ERM minimizes training error; the two coincide only as $n \rightarrow \infty$.

Generative ($P(X|Y)P(Y)$ then Bayes) GDA, LDA, QDA, Naive Bayes. Models class-conditional density of features.

Discriminative ($P(Y|X)$ directly) Logistic regression, neural networks, SVM, AdaBoost, decision trees. **AdaBoost can return $P(Y = C \mid x) \approx \sigma(2M(x))$ but is still discriminative — it never models $P(X|Y)$.**

Logistic regression

Discriminative probabilistic model $p(y|X)$ (directly; model decision boundary) e.g. $p(y = C|X, w) = \sigma(w^T X)$

- Sigmoid identities

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$
$$\frac{\sigma(z)}{1 - \sigma(z)} = e^z \quad \Rightarrow \quad \log \frac{\sigma(z)}{1 - \sigma(z)} = z$$

- Binary logistic model

$$p(y = 1 \mid x; w) = \sigma(w^T x), \quad p(y = 0 \mid x; w) = 1 - \sigma(w^T x)$$

i.e. log-odds are linear in x :

$$\log \frac{p(y = 1 \mid x; w)}{p(y = 0 \mid x; w)} = w^T x$$

- Cross-entropy/NLL For data $\{(x_i, y_i)\}_{i=1}^n$, $y_i \in \{0, 1\}$:

$$J(w) = - \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

where $\hat{p}_i = \sigma(w^T x_i)$.

- Logistic regression gradient/Hessian Let $X \in \mathbb{R}^{n \times d}$, $p = \sigma(Xw)$ elementwise, and $S = \text{diag}(p_i(1 - p_i))$. Then

$$\nabla_w J(w) = X^T(p - y), \quad \nabla_w^2 J(w) = X^T S X$$

Since $S \succeq 0$, logistic NLL is convex.

Class-conditional gaussians

Generative probabilistic model $p(y|X) \propto p(X, y) = p(X|y)p(y)$ (indirectly via Bayes Rule; fully model the density X of each class i.e. DGP) e.g. $p(X|y = C, \mu_C, \Sigma_C) = \mathcal{N}(X|\mu_C, \Sigma_C)$ where $p(y = 1) = 0.22, p(y = 0) = 0.78$

LDA different means, same covariance matrix; ellipsoid contours with same shape/orientation but centered at different means

QDA different means and different covariance matrices

GDA each class generates data according to a Gaussian distribution $p(x|y = k) = \mathcal{N}(x|\mu_k, \Sigma_k)$, predict MAP $\hat{y} = \arg \max_k \log p(x|y = k) + \log p(y = k)$

GDA / LDA / QDA MLE estimates Let $n_k = |\{i : y_i = k\}|$. Then

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i$$

For QDA:

$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

For LDA (pooled covariance):

$$\hat{\Sigma} = \frac{1}{n} \sum_{k=1}^K \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

ROC curve

Algorithm:

1. Sort the test instances by decreasing score
2. Move down the list (lowering the threshold), processing one instance at a time
3. Compute the TPR and FPR and add one point to the plot

ROC curve property: any instance that is classified as + at a given threshold, will be classified as + for all lower thresholds as well. **Note: increase threshold $\Rightarrow$ TPR and FPR decrease, precision anything**

$TPR = \frac{TP}{TP+FN}$ is recall i.e. sensitivity, $TNR = \frac{TN}{TN+FP}$ i.e. specificity, $FPR = \frac{FP}{FP+TN}$.

Precision is equal $\frac{TP}{TP+FP}$ i.e. depends on class distribution in set of points being evaluated.

Likelihood/log-likelihood Given $x_1, \dots, x_n \stackrel{\text{iid}}{\sim} p(x | \theta)$:

$$L(\theta) = p(x_1, \dots, x_n | \theta) = \prod_{i=1}^n p(x_i | \theta), \quad \ell(\theta) = \log L(\theta) = \sum_{i=1}^n \log p(x_i | \theta)$$

MLE

$$\hat{\theta}_{\text{MLE}} \in \arg \max_{\theta} L(\theta) = \arg \max_{\theta} \ell(\theta)$$

least squares regression under Gaussian noise:

$$p(y \mid x, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(x))^2}{2\sigma^2}\right)$$

s.t. $y = f(x) + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$.

MAP

$$\hat{\theta}_{\text{MAP}} \in \arg \max_{\theta} p(\theta \mid \mathcal{D}) = \arg \max_{\theta} (\ell(\theta) + \log p(\theta))$$

MAP (ridge) ridge regression with a zero-mean Gaussian prior:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)p(\theta), \quad p(\theta) \sim \mathcal{N}(0, \tau^2 I)$$

MAP (LASSO) least squares with a zero-mean Laplace prior:

$$p(\theta_j) \propto \exp(-\lambda|\theta_j|)$$

i.e. maximizing the log-posterior is equivalent to minimizing

$$\sum_{i=1}^n (y_i - x_i^T \theta)^2 + c \|\theta\|_1$$

Gaussian MLE If $x_1, \dots, x_n \stackrel{\text{iid}}{\sim} \mathcal{N}(\mu, \Sigma)$, then

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

Hessian Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$

$$\nabla^2 f(x) = \begin{bmatrix} \frac{d^2}{dx_1^2} f(x) & \dots & \frac{d^2}{dx_1 dx_n} f(x) \\ \dots & & \\ \frac{d^2}{dx_n dx_1} f(x) & \dots & \dots \frac{d^2}{dx_n^2} f(x) \end{bmatrix}$$

$$f(x) = x^T A x := \sum_{i=1}^n x_i (A x)_i = \sum_{i=j} x_i \left(\sum_{j=1}^n A_{ij} x_j \right) = \sum_{i=j} A_{ii} x_i^2 + \sum_{i \neq j} A_{ij} x_i x_j$$

$$\frac{d}{dx} x^T A x := (A^T + A)x = 2 \sum_{i=j} A_{jj} x_j + \sum_{i \neq j} (A_{ij} + A_{ji}) x_j = \sum_{i=1}^n A_{ij} x_j + \sum_{i=1}^n A_{ji} x_j$$

$$\frac{d^2}{dx^2} x^T A x = A + A^T$$

Gradient descent

- Guaranteed convergence if:

1. Convex: $\nabla^2 f \succeq 0$ i.e. Hessian of objective function is PSD
 2. L-smooth: $\|\nabla^2 f\| = \lambda_{\max} \leq L \forall x$
 3. Learning rate $\mu \leq 1/L$ (in class: $\mu < 2/\lambda_{\max}$)
- *Batch gradient descent algorithm.* Update weights by moving in the opposite direction of the gradient computed over *all* N training points:

init $w^{(0)}$
 for step $\tau = 1, \dots, \tau_{\max}$:
 $w^{(\tau)} = w^{(\tau-1)} - \eta \nabla J(w^{(\tau-1)})$
 if converged($w^{(\tau)}, w^{(\tau-1)}, \epsilon$) : terminate

s.t. $\nabla J(w) = \frac{1}{N} \sum_{n=1}^N \nabla L(t_n, f_w(x_n))$. Cost per update is $O(Nd)$ for N data points in d dimensions.

- *Convergence analysis.*

The 2nd-order (quadratic) Taylor approximation of a function $f : \mathbb{R} \rightarrow \mathbb{R}$ evaluated at $\hat{x}$:

$$\tilde{f}(x) = f(\hat{x}) + f'(\hat{x})(x - \hat{x}) + \frac{1}{2} f''(\hat{x})(x - \hat{x})^2$$

The 2nd-order Taylor expansion of the error surface at $\hat{w}$:

$$\tilde{E}_{\hat{w}}(w) = E(\hat{w}) + (w - \hat{w})^T \cdot \nabla E(\hat{w}) + \frac{1}{2} (w - \hat{w})^T \cdot \nabla^2 E(\hat{w}) \cdot (w - \hat{w})$$

s.t. if $\hat{w}$ is critical point w^* , then $\nabla E(w^*) = 0 = \nabla E(\hat{w})$

Eigendecomposition of the Hessian:

$Hu_i = \lambda_i u_i$ s.t. u_i : orthonormal eigenvectors $\forall i, j : u_i^T u_j = \delta_{ij}$.

Change of variables: $w - w^* = \sum_i \alpha_i u_i$ where $\alpha_i = (w - w^*)^T u_i$.

Quadratic approximation error:

$$E_{w^*}(\alpha) = E(w^*) + \frac{1}{2} \sum_i \alpha_i^2 \lambda_i; \quad \frac{d}{d\alpha_i} E_{w^*}(\alpha) = \alpha_i \lambda_i$$

i.e. minimized at $\alpha = 0$, we get error at the critical point $E(w^*)$.

If we move in direction i (corresponding to u_i ; i.e. $w = w^* + t u_i$), then α_i increases. The change in error depends on the sign of λ_i .

At critical point, if $H \succ 0$: w^* local min, $H \prec 0$: w^* local max, mixed: w^* saddle point; i.e. eigenvalues of the Hessian determine the *local* curvature at critical point.

Gradient descent update: $\alpha_i^{(\tau)} = \alpha_i^{(\tau-1)} - \eta(\alpha_i^{(\tau-1)} \lambda_i) = (1 - \eta \lambda_i) \alpha_i^{(\tau-1)} = (1 - \eta \lambda_i)^\tau \alpha_i^{(0)}$ i.e. to ensure $\alpha_i^{(\tau)} \rightarrow 0$ as $\tau \rightarrow \infty$ the learning rate should satisfy $|1 - \mu \lambda_i| < 1$. ($1 - \mu \lambda_i < 0$: oscillations.)

If $\eta \geq \frac{2}{\lambda_{\max}}$, then $|1 - \eta \lambda_{\max}| \geq 1$. Then $\eta < \frac{2}{\lambda_{\max}}$.

Condition number: high $\lambda_{\max}/\lambda_{\min}$ mean slow convergence.

The slowest converging dimension i is the dimension corresponding to $\lambda_{\min}$ and its convergence to 0 is lower bounded by $\left(1 - \frac{2}{\lambda_{\max}}\lambda_{\min}\right) < (1 - \eta\lambda_i)$.

• Issues with GD:

- If error surface is too flat, slow convergence even with reasonable learning rate; **use Momentum**
- If error surface is ill-conditioned (too big condition number $\frac{\lambda_{\max}}{\lambda_{\min}}$), learning rate that works for shallow direction (small gradient) overshoots for steep direction (big gradient) resulting in **zigzags**; **use AdaGrad or RMSProp**

Momentum accumulate common update directions $\Delta w^{(\tau-1)} = -\eta \nabla J(w^{(\tau-1)}) + \mu \Delta w^{(\tau-2)}$

$$w^{(\tau)} = w^{(\tau-1)} - \eta \nabla J(w^{(\tau-1)}) + \mu(w^{(\tau-1)} - w^{(\tau-2)}) \quad \mu \in [0, 1) : \text{hyperparam}$$

s.t. in directions that are relatively flat $\Delta w^{(\infty)} = -\eta \nabla J(1 + \mu + \mu^2 + \dots) = -\frac{\eta}{1-\mu} \nabla J$ s.t. $\eta \leq \frac{\eta}{1-\mu}$ and in directions that are steep $\Delta w^{(\infty)} = -\eta \nabla J(1 - \mu + \mu^2 - \dots) = -\frac{\eta}{1+\mu} \nabla J$ s.t. $\frac{\eta}{1+\mu} \leq \eta$

Learning rate scheduling reducing noise in the beginning and finetuning towards the end **lr ↘**

- **Linear**: interpolates $\eta_{\text{start}} \rightarrow \eta_{\text{end}}$ over T iterations $\eta^{(\tau)} = \left(1 - \frac{\tau}{T}\right) \eta_{\text{start}} + \frac{\tau}{T} \eta_{\text{end}}$
- **Power-law**: (most common) with $\eta_{\text{start}}, s, c$ hyperparameters $\eta^{(\tau)} = \eta_{\text{start}} \left(1 + \frac{\tau}{s}\right)^{-c}$
- **Exp**: with $\eta_{\text{start}}, s, c$ hyperparameters $\eta^{(\tau)} = \eta_{\text{start}} c^{\lfloor \tau/s \rfloor}$
- **Cos**: common for LLMs, with $\eta_{\min}, \eta_{\max}, T$ hyperparameters $\eta^{(\tau)} = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\tau}{T}\pi\right)\right)$

AdaGrad reduce lr on dimensions with greatest curvature (largest gradient magnitude) using running squared average of the gradient

$$r_i^{(\tau)} = r_i^{(\tau-1)} + \left(\frac{d}{dw_i} J(w)\right)^2, \quad w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \epsilon}} \left(\frac{d}{dw_i} J(w)\right)$$

issue: with large initial gradients, result in overly small lr near convergence

RMSProp focus on most recent gradients using exponential weighted average

$$r_i^{(\tau)} = \beta r_i^{(\tau-1)} + (1 - \beta) \left(\frac{d}{dw_i} J(w)\right)^2$$

s.t. init $r_i^{(0)} = 0$. Typically choose hyperparam $\beta = 0.9$.

Adam RMSProp + Momentum (*adaptive moments*)

Init: $s_i^{(0)} = 0$ and $r_i^{(0)} = 0$.

$$s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1 - \beta_1) \frac{d}{dw_i} J(w^{(\tau-1)}), \quad r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1 - \beta_2) \left(\frac{d}{dw_i} J(w^{(\tau-1)}) \right)^2$$

Bias correction: since $s_i^{(0)} = r_i^{(0)} = 0$, early estimates are biased toward 0.

$$\hat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1 - \beta_1^\tau}, \quad \hat{r}_i^{(\tau)} = \frac{r_i^{(\tau)}}{1 - \beta_2^\tau}$$

Parameter update:

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \frac{\hat{s}_i^{(\tau)}}{\sqrt{\hat{r}_i^{(\tau)} + \epsilon}}$$

- $\hat{s}_i^{(\tau)}$: smoothed first moment of gradient (*direction*)
- $\hat{r}_i^{(\tau)}$: smoothed second moment of gradient magnitude (*scale*)
- divide by $\sqrt{\hat{r}_i^{(\tau)}}$ to reduce effective learning rate in coordinates with consistently large gradients

Typically choose hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Adam addresses:

- noisy/sparse gradients via momentum averaging
- ill-conditioned coordinates via per-coordinate adaptive step sizes
- overly small early steps via bias correction

Weighted least squares For $A = \text{diag}(\alpha_1, \dots, \alpha_n)$, $\alpha_i \geq 0$:

$$J(w) = (y - Xw)^T A (y - Xw)$$

$$\nabla_w J(w) = -2X^T A y + 2X^T A X w$$

$$w^* = (X^T A X)^{-1} X^T A y$$

Ridge regression

$$J(w) = \|y - Xw\|_2^2 + \lambda \|w\|_2^2, \quad \lambda > 0$$

$$\nabla_w J(w) = 2(X^T X + \lambda I)w - 2X^T y$$

$$w^* = (X^T X + \lambda I)^{-1} X^T y$$

s.t. $v^T (X^T X + \lambda I) v = \|Xv\|_2^2 + \lambda \|v\|_2^2 > 0 \quad \forall v \neq 0$.

Bias-variance tradeoff

$$\mathbb{E}[(t - f_D(x))^2] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{(h(x) - \mathbb{E}[f_D(x)])^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(f_D(x) - \mathbb{E}[f_D(x)])^2]}_{\text{variance}}$$

More complex model: bias ↓, variance ↑. Stronger regularization: bias ↑, variance ↓.

Entropy/cross-entropy/KL

$$H(p) = - \sum_x p(x) \log p(x)$$

$$H(p, q) = - \sum_x p(x) \log q(x)$$

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0$$

$$H(p, q) = H(p) + D_{\text{KL}}(p||q)$$

AUC = $\Pr(S^+ > S^-) + \frac{1}{2} \Pr(S^+ = S^-)$ i.e. probability that random positive is ranked above random negative.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

i.e. fraction of variance explained relative to predicting the mean.

PCA Objective with reconstruction error:

$$\mathcal{L}(W, Z) = \frac{1}{N} \|X - ZW^T\|_F^2 = \frac{1}{N} \sum_{n=1}^N \|x_n - Wz_n\|^2$$

s.t. $X \in \mathbb{R}^{N \times D}$: data matrix, $W \in \mathbb{R}^{D \times L}$: matrix of latent factors, $Z \in \mathbb{R}^{N \times L}$

Decision Trees, Entropy & Information Gain

Entropy For node with class proportions p_C :

$$H = - \sum_C p_C \log_2 p_C, \quad 0 \leq H \leq \log_2 K$$

$H = 0$ at purity ($p_C = 1$); $H = \log_2 K$ at uniform. Strictly concave in p ; defined for any $K \geq 2$.

Information Gain (IG)

$$\text{IG} = H(\text{parent}) - (w_L H(\text{left}) + w_R H(\text{right}))$$

where w_L, w_R are the fractions of points sent to each child.

- $\text{IG} \geq 0$ always (Jensen + concavity).
- $\text{IG} = 0$ iff both children have the same class distribution as the parent (or one child is empty).
- Concavity is what makes mixed nodes “expensive” and pure nodes “cheap”.

Alternatives

- Gini: $G = 1 - \sum_C p_C^2$. Strictly concave, like entropy. CART default.
- Misclassification error: $1 - \max_C p_C$. **Piecewise linear, NOT strictly concave** — bad splitting criterion (gives IG = 0 for splits that are clearly purer when the majority class doesn't change).
- Gain ratio (C4.5): IG/split entropy — corrects bias toward features with many values.

Common traps: IG can never be negative; entropy works for any number of classes; trees grown to purity overfit (train error = 0 but val error high) — fix with pruning, max depth, min samples/leaf.

CNN

Local connectivity: each neuron sees only a local patch of the input. Weight sharing: the same filter is applied across all spatial locations. Each conv layer computes:

$$h_{i,j,k} = \sum_{u,v,c} W_{u,v,c}^{(k)} \cdot x_{i+u,j+v,c} + b^{(k)} \quad \textcolor{red}{k : \text{output channel index, } (u,v) : \text{kernel spatial dims index}}$$

2D conv: $X' = K * X$

Output spatial size

$$N_{\text{out}} = \left\lfloor \frac{N + 2P - F}{S} \right\rfloor + 1$$

Params per conv layer $F_h F_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ (bias). Independent of input spatial size due to weight sharing.

Inductive biases Translation equivariance: shifting input shifts feature map. Locality: nearby pixels are more correlated. Parameter efficiency: fewer parameters due weight sharing. **All improve sample efficiency w.r.t. FC.**

Pooling 0 trainable params. Changes spatial dims: $(C, H, W) \rightarrow (C, H/k, W/k)$ for $k \times k$ stride- k . Channel count unchanged. **Improve invariance to small translations, reduce computation, increase effective receptive field.**

Receptive field of a neuron is the region of input that influences its value; grows with depth.

$$j_0 = r_0 = 1, \quad j_\ell = \textcolor{red}{j_{\ell-1}} s_\ell, \quad r_\ell = r_{\ell-1} + (k_\ell - 1) \textcolor{red}{j_{\ell-1}}$$

Two stacked 3×3 convs $\equiv$ one 5×5 in receptive field, with fewer params and more nonlinearity. Each convolution expands receptive field linearly (or multiplicatively with stride). Deep networks allow global context capture from local operations.

Conv variants. **Depthwise:** each filter attends to one input channel (no cross-channel mixing); params $C_{\text{in}} k_H k_W + C_{\text{in}}$. Used in MobileNet, ConvNeXt. **Depthwise separable** (depthwise + 1×1 pointwise): params $C_{\text{in}} k_H k_W + C_{\text{in}} C_{\text{out}}$; much cheaper than full conv when $k_H k_W$ is large. **Grouped** (e.g., ResNeXt): channels split into g groups, each filter sees only its group; params $\frac{C_{\text{in}}}{g} k_H k_W C_{\text{out}} + C_{\text{out}}$.

Complexity. Per conv layer: $O(k \cdot n \cdot d^2)$ where $n = HW$ is spatial size, d is channel width, k is kernel size. Sequential ops $O(1)$ (parallelizable across positions); maximum path length $O(\log_k n)$ — receptive field grows logarithmically with depth for stacked convs, so connecting distant pixels requires $\log_k n$ layers.

Layer Type	Complexity per Layer	Sequential Ops	Max Path Length
Self-Attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\log_k n)$
Self-Attention (restricted)	$O(rnd)$	$O(1)$	$O(n/r)$

Common 2D convolution kernels blur/avg $\Rightarrow$ all ≥ 0 , sum = 1; edge/derivative $\Rightarrow$ mixed signs, sum = 0; sharpen $\Rightarrow$ identity + edge, sum = 1.

- **Identity** $K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

- **Box blur** $K = \frac{1}{4} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$. Avg pool when stride = kernel size.

- **Gaussian blur** $K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$, samples $\exp(-(x^2 + y^2)/2\sigma^2)$.

- **Vertical edge** ($\partial I / \partial x$, fires on left $\rightarrow$ right change): $S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$.

Horizontal edge ($\partial I / \partial y$, fires on top $\rightarrow$ bottom change): $S_y = S_x^T = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$.

Edge magnitude (orientation-agnostic): $\sqrt{(S_x * I)^2 + (S_y * I)^2} \approx \|\nabla I\|$.

- **Laplacian** $L = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, approximates $\nabla^2 I$. Orientation-free edge/blob detector. *Sobel/Prewitt* =

smoothed first-derivative variants (Prewitt: uniform rows; Sobel: center row $\times 2$, less noise).

- **Sharpen** $K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} = \text{identity} + L$. Enhances local contrast.

Early CNN layers learn filters resembling these (Gabor-like edges, blobs); deeper layers compose them.

Spatial transformation matrices (image augmentation) For flattened image $\text{vec}(I) \in \mathbb{R}^N$, $N = HW$, an augmentation is $T \cdot \text{vec}(I)$ for permutation matrix $T \in \{0, 1\}^{N \times N}$ (1 nonzero per row). Composable: $T_{\text{total}} = T_k \cdots T_1$.

- **Vertical flip:** $T[(H-1-i)W+j, iW+j] = 1$.
- **Horizontal flip:** $T[iW+(W-1-j), iW+j] = 1$.
- **Shift** by (dx, dy) : $T[iW+j, (i-dy)W+(j-dx)] = 1$ if source in bounds, else 0 (pixels falling off the canvas are lost $\Rightarrow$ shift is not invertible in general).
- **Rotation** by θ : rotate $(j-c_j, i-c_i)$ by θ , round to nearest pixel. Bilinear-interp version uses 4 fractional weights (not 0/1) for smoother rotation.

Flips: always invertible ($T^2 = I$, so $T^{-1} = T$). Rotations: invertible if no off-canvas loss + interpolation handled. Shifts/blur: information loss $\Rightarrow$ generally not invertible. Distinct from conv kernels: these are $N \times N$ *geometric* (per-image), not small *local* filters — used for data augmentation, not feature extraction.

ResNets

Degradation problem deeper plain nets have *worse training error* than shallower — optimization failure, not overfitting. (Identity emulation is theoretically possible but hard to learn.)

Residual block $y = x + \mathcal{F}(x; W)$. If identity is optimal, set $\mathcal{F} = 0$ — much easier than learning identity through stacked layers.

Dimension mismatch fixes (a) zero-padding extra channels (param-free); (b) projection shortcut $y = W_s x + \mathcal{F}(x)$ via 1×1 conv.

Attention / Transformers

How weights used soft attention: weigh over whole vocab, hard attention: sample

How weight computed location based: rnn, content based: transformers

Scaled dot-product attention $Q = XW_Q$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

Q : what each token is looking for, K : what each token offers, V : the information being aggregated Scaling by $\sqrt{d_k}$ keeps $\text{Var}(q^T k) = 1$ when $q, k \sim \mathcal{N}(0, I_{d_k})$, preventing softmax saturation.

QKV sources

- Encoder self-attn: Q, K, V from encoder.
- Decoder self-attn: Q, K, V from decoder prefix (**causal mask**).
- Cross-attn: Q from decoder, K, V from encoder output.

Causal mask $M_{ij} = 0$ if $j \leq i$, else $-\infty$. Enables parallel training while preventing tokens from attending to the future.

Softmax properties Temperature with $\text{softmax}(z/T)$: $T \rightarrow 0$ one-hot; $T \rightarrow \infty$ uniform.

- invariant to adding a constant: $\text{softmax}(z) = \text{softmax}(z + c)$
- large inputs lead to *peaky* distributions (hard attention)
- small inputs lead to *uniform* distributions

Ideal positional encoding s.t. $X' = X + P$:

1. provide a unique representation $r_n \in \mathbb{R}^D$ for each location
2. be bounded on the scale of the original features x_n (for stable training)
3. express relative distances between tokens (inductive bias: relative location is important for images and language)
4. should work for arbitrary number of tokens N

RoPE $\text{RoPE}(x, n) = R(n\theta)x$. Then

$$\text{RoPE}(x, m)^T \text{RoPE}(y, n) = x^T R((n - m)\theta)y$$

depends only on relative position $n - m$. Invariant to global shift of all positions.

KV cache Without cache: at step n , recompute $K_{1:n}, V_{1:n}$ — total $O(N^2)$ projections over a sequence of length N . With cache: each K_i, V_i computed once — total $O(N)$. Critical for chatbots / long contexts.

ViT Images are treated as sequences of $p \times p$ patches; flatten each patch into a vector, linearly project into embeddings

$$X \in \mathbb{R}^{N \times d}, \quad N = \frac{HW}{p^2}$$

- no convolutional inductive bias
- global receptive field from first layer
- requires more data but highly expressive

Normalization: BatchNorm vs LayerNorm

For input $X \in \mathbb{R}^{B \times n_l}$ (rows = datapoints, cols = features):

LayerNorm (per datapoint, sum over features)

$$\mu_i = \frac{1}{n_l} \sum_{j=1}^{n_l} x_{i,j}, \quad y_{i,j} = \gamma_j \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_j$$

Produces B different (μ, σ) pairs. Independent of batch size; same at train/test.

BatchNorm (per feature, sum over batch)

$$\mu_j = \frac{1}{B} \sum_{i=1}^B x_{i,j}, \quad y_{i,j} = \gamma_j \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j$$

Produces n_l different (μ, σ) pairs. Depends on batch size; uses running stats at inference.

BN normalizes columns, LN normalizes rows. LN is used in transformers because it's batch-size independent and identical train/test.

RMSNorm like LayerNorm without mean subtraction:

$$y = \gamma \cdot \frac{x}{\sqrt{\frac{1}{D} \sum_j x_j^2 + \epsilon}}$$

Cheaper than LN; standard in modern LLMs (e.g. LLaMA).

Optimizers & Weight Decay (Adam vs AdamW)

SGD with momentum $v_t = \mu v_{t-1} + \nabla L$, $w_t = w_{t-1} - \eta v_t$. Larger μ : smoother, more directional trajectory.

Adam (L2-coupled weight decay) adds $\lambda \|w\|^2$ to loss $\Rightarrow \lambda w$ goes through $\sqrt{\hat{v}}$ adaptive normalization. Decay strength becomes coupled to gradient history.

AdamW (decoupled)

$$w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}} - \eta \lambda w$$

Decay applied directly to weights, outside adaptive normalization. Larger λ pulls trajectory toward origin in weight space — preferred for transformers.

Batch size larger $\Rightarrow$ smoother gradient curve (less noise), better data parallelism. Does NOT reduce memory (more activations).

Learning rate smaller $\Rightarrow$ smoother monotonic descent but more steps to converge; same per-step cost.

Batch size / LR scaling heuristics when batch size b increases by factor k :

- **Linear scaling:** $\eta \leftarrow k\eta$ (with warmup to prevent early instability).
- **Square-root scaling:** $\eta \leftarrow \sqrt{k}\eta$ — safer when gradient noise matters.

LR too large: overshoots / diverges. **LR too small:** slow / plateau. Warmup prevents unstable early steps in deep nets / Transformers.

Training diagnostics

- **Train high / val high:** underfit / high bias. Fix: more capacity, train longer, less regularization.
- **Train low / val high:** overfit / high variance. Fix: data/augmentation, dropout, weight decay, early stopping, smaller model.
- **Loss diverges:** LR too high or exploding gradients. Fix: lower LR, gradient clipping.
- **Loss plateaus:** LR too small, bad init, dead ReLUs, saturation.
- **Noisy loss:** small batch / high stochasticity.

Small vs large batch

- **Small batch:** noisy gradients, cheap updates, regularizing noise, less memory, worse hardware utilization.
- **Large batch:** accurate gradients, faster wall-clock on GPU/TPU, less noise, may need LR scaling/warmup, can generalize worse if too large.

Curse of Dimensionality Volume of $[0, 1]^d$ grows exponentially in dimension. To cover $[0, 1]^d$ with grid spacing h requires $(1/h)^d$ cells (e.g. spacing 0.1 needs 10^d samples). If n points are uniform in $[0, 1]^d$, typical nearest-neighbor distance scales like $n^{-1/d}$, so data becomes sparse and distances concentrate. **Hurts:** kNN, kernel methods, fixed polynomial bases, density estimation. **Deep learning helps:** learns low-dimensional / manifold features rather than fixed bases in raw ambient dimension.

Universal approximation caveats Universal approximation is an existence result, not a learnability / sample-complexity guarantee. Depth can represent some functions exponentially more efficiently than width.

Capacity control Capacity: ability of a model to fit a wide range of functions. High capacity models can represent very complex functions (and potentially memorize data). **Reduce capacity with a structural change only.**

Reduce effective model complexity: reduce parameters, early stop, dropout (force model rely on multiple redundant representations, prevent co-adaptation: neurons relying on each other in a highly specific way to produce correct outputs).

Self-Supervised Learning & Contrastive Loss

SSL overview Create supervision from raw data: pretext task trains representation, downstream task uses / fine-tunes it.

Transfer learning

- Small dataset: freeze feature extractor, train classifier head only.
- Larger dataset: fine-tune more layers with low LR.

Challenge: choosing pretext / loss and evaluating representation quality.

Autoencoders $z = f_\theta(x)$, $\hat{x} = g_\phi(z)$, $L = \|x - \hat{x}\|_2^2$. Bottleneck / regularization forces compression.

- **Denoising AE**: reconstruct clean x from corrupted $\tilde{x}$.
- **Sparse AE**: penalize dense latents via $\mathcal{L}_{\text{Sparse}}(z) = \|z\|_1$ (convex relaxation of $\|z\|_0$).

L2 shrinks but doesn't zero out. Cannot achieve $\mathcal{L}_{\text{Recon}} = 0 = \mathcal{L}_{\text{Sparse}}$ simultaneously on distinct images — sparse $z = 0$ kills decoder info.

Pretext tasks

- Colorization / cross-channel prediction.
- Super-resolution.
- Rotation classification (discriminative, CE).
- Jigsaw / context prediction.
- Clustering pseudo-labels.
- Autoencoder (generative, MSE), SimCLR (discriminative, contrastive).

Context encoder / inpainting mask part of input and predict missing region:

$$\mathcal{L} = \lambda_{\text{rec}} \|M \odot (x - \hat{x})\|_2^2 + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}$$

MSE alone $\Rightarrow$ blurry (averages plausible completions); adversarial loss encourages realistic / sharp completions.

InfoNCE For reference i with positive z_i^+ and negatives z_j^- :

$$\mathcal{L}_i = -\log \frac{f(z_i^+, c_i)}{f(z_i^+, c_i) + \sum_{j=1}^{N-1} f(z_j^-, c_i)}, \quad f(z, c) = \exp(z^T W c)$$

Interpretation: N -way classification — pick the true positive among N candidates.

Gradient w.r.t. similarity score $s_j = z_j^T W c_i$

$$\frac{\partial \mathcal{L}_i}{\partial s_j} = p_j - \mathbb{1}[j = +]$$

where $p_j = \text{softmax}(s)_j$. Positive: $p_+ - 1 < 0 \Rightarrow$ pushed up. Negatives: $p_j > 0 \Rightarrow$ pushed down.

Effect of N Larger N : tighter lower bound on mutual info, better representations. **Tradeoff: memory/compute, more false negatives.**

Why contrastive over reconstruction avoids modeling pixel-level detail; latent space focused on semantic similarity, not unimportant variation.

Pretext tasks summary autoencoder (gen, MSE), context encoder (gen, MSE+adv), rotation (disc, CE), SimCLR (disc, contrastive).

InfoNCE for biology (scRNA-seq) $x_i \in \mathbb{R}^d$ gene expression, $z_i = g_{\text{enc}}(x_i)$.

- Positive: dropout augmentation of same cell, or biologically nearby cell.
- Negatives: unrelated cells.

With context c_i and $f(z, c_i) = \exp(z^T W c_i)$:

$$\mathcal{L}_i = -\log \frac{f(z_i^+, c_i)}{f(z_i^+, c_i) + \sum_{j=1}^{N-1} f(z_j^-, c_i)}$$

Interpret as N -way classification: identify the biologically related candidate.

Diffusion & Flow Matching

Forward noising $x_\sigma = x_0 + \sigma \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$.

Conditional expectation minimizer For random X, Y :

$$\arg \min_f \mathbb{E} \|f(Y) - X\|^2 = \mathbb{E}[X \mid Y]$$

Optimal denoiser $\epsilon_\sigma^*(x_\sigma) = \mathbb{E}[\epsilon \mid x_\sigma]$. Equivalent to score: $\nabla_x \log p_\sigma(x) = -\mathbb{E}[\epsilon \mid x_\sigma]/\sigma$.

Continuity equation $\partial_t p_t(x) + \nabla \cdot (v(x, t) p_t(x)) = 0$ for velocity $v(x, t) = \mathbb{E}[\frac{d}{dt} \sigma(t) \epsilon \mid x_t = x]$.

Flow matching path $x_t = \alpha(t)x_1 + \beta(t)x_0$ with $\alpha(0) = 0, \beta(0) = 1, \alpha(1) = 1, \beta(1) = 0$.

$$\dot{x}_t = \alpha'(t)x_1 + \beta'(t)x_0$$

For linear path $\alpha = t, \beta = 1 - t$: $\dot{x}_t = x_1 - x_0$ (constant).

Single-function path $x_t = h(t)x_1 + (1 - h(t))x_0$ with h monotone, $h(0) = 0, h(1) = 1$:

$$\dot{x}_t = h'(t)(x_1 - x_0)$$

- $h(t) = t$: constant speed.
- $h(t) = t^2$: $h'(t) = 2t$ — slow near $t = 0$, fast near $t = 1$.
- $h(t) = \sin(\pi t/2)$: $h'(t) = (\pi/2) \cos(\pi t/2)$ — fast near $t = 0$, slow near $t = 1$.

Diffusion frequency progression early steps ($t \approx T$): low-frequency / global structure. Late steps ($t \approx 0$): high-frequency / fine details. [Coarse-to-fine generation](#).

AR vs Diffusion training efficiency both decouple deep inference path from per-step backprop — train on single-step objective (next-token CE / single- t noise prediction), avoiding gradients through full unrolled chain.

Vision-Language Models, Agents, GEPA

VLM architecture vision encoder + projection / resampler + LLM. Vision embeddings $H_{\text{vis}} \in \mathbb{R}^{m \times D_{\text{vis}}}$ must be projected to text embedding size D_{txt} .

Projection layer For a single visual vector $h \in \mathbb{R}^{D_{\text{vis}}}$:

$$W_P \in \mathbb{R}^{D_{\text{txt}} \times D_{\text{vis}}}, \quad W_P h \in \mathbb{R}^{D_{\text{txt}}}$$

E.g. $D_{\text{vis}} = 1024, D_{\text{txt}} = 4096 \Rightarrow W_P$ is 4096×1024 .

Why freeze backbones during alignment

- Prevent catastrophic forgetting of pretrained vision / language capabilities.
- Save VRAM / compute.

Train projection with image-text SFT / teacher-forced CE.

[CLS] vs patch tokens CLS-only: efficient, loses spatial detail. Patch tokens: preserve grounding, expensive. Modern VLMs use full patch sequences.

Special tokens CLS, SEP, BOS/EOS, PAD, image placeholders.

LoRA $W' = W + AB$, $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$, $r \ll d$. Train only A, B .

Pareto frontier Set of solutions where no other solution dominates (not strictly better on every objective). In GEPA: prompts that excel on at least one val item — keeps diversity, prevents premature collapse.

GEPA loop (1) split train into dev/val; (2) maintain pool incl. Pareto front; (3) repeatedly: select prompt from frontier, run on dev minibatch, ask LM to mutate/crossover, update pool; (4) finally pick best on average.

Universal Approximation & GD Convergence

A feedforward nn with a single hidden layer and a nonlinear activation function can approximate any continuous function arbitrarily well.

Universal approximation construction For continuous $f : [0, 1] \rightarrow \mathbb{R}$ and $\epsilon > 0$, partition $0 = x_0 < \dots < x_n = 1$ with $x_k - x_{k-1} < \delta$ (uniform continuity). Linear interpolant:

$$F(x) = C + m_1 \text{ReLU}(x) + \sum_{k=1}^{n-1} (m_{k+1} - m_k) \text{ReLU}(x - x_k)$$

matches f on each kink, slopes $m_k = (f(x_k) - f(x_{k-1})) / (x_k - x_{k-1})$. Then $\max_x |f - F| < \epsilon$.

Strongly convex GD convergence If $\mu I \preceq \nabla^2 f \preceq LI$ and $0 < \alpha < 2/L$:

$$\|x_k - x^*\|_2 \leq M^k \|x_0 - x^*\|_2, \quad M = \max\{|1 - \alpha\mu|, |1 - \alpha L|\} < 1$$

Optimal step: $\alpha^* = \frac{2}{L+\mu}$, contraction $M^* = \frac{\kappa-1}{\kappa+1}$, $\kappa = L/\mu$.

Iteration complexity $k = O(\log(1/\epsilon))$ iterations to achieve $\|x_k - x^*\| \leq \epsilon$ — *linear convergence*.

Backprop & Activation

Computation graph $z \rightarrow z = Wx \rightarrow h = \text{ReLU}(z) \rightarrow y = v^T h \rightarrow \mathcal{L}$

$$\frac{dL}{dW} = \frac{dL}{dy} \frac{dy}{dh} \frac{dh}{dW} = 2 \frac{1}{2} (y - y^*) v \odot \mathbb{1}_{x>0} x^T$$

$$\frac{dL_{\text{reg}}}{dW} = \frac{dL}{dW} + \lambda W$$

Softmax + cross-entropy gradient

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \text{ i.e. } p = \text{softmax}(z) \text{ s.t. } z = w^T x, \quad L = - \sum_{k=1}^K y_k \log p_k, \quad \frac{\partial L}{\partial z} = p - y$$

Stable softmax: subtract $\max_j z_j$ from z_k before exponentiating.

Binary cross-entropy $p = \sigma(w^T x)$, $L = -y \log p - (1 - y) \log(1 - p)$.

Matrix backprop For $Y = XW$ with upstream $G = \frac{dL}{dY}$:

$$\frac{\partial L}{\partial X} = GW^T, \quad \frac{\partial L}{\partial W} = X^T G$$

Affine $y = Wx + b$: $\bar{x} = W^T \bar{y}$, $\bar{W} = \bar{y} x^T$, $\bar{b} = \bar{y}$. Elementwise $h = \phi(a)$: $\bar{a} = \bar{h} \odot \phi'(a)$.

One-hidden-layer MSE $z = Wx$, $h = \text{ReLU}(z)$, $\hat{y} = v^T h$, $L = \frac{1}{2}(\hat{y} - y)^2$:

$$\partial L / \partial v = (\hat{y} - y)h, \quad \delta = (\hat{y} - y)v \odot \mathbb{1}[z > 0], \quad \partial L / \partial W = \delta x^T$$

Inactive ReLU $\Rightarrow$ zero gradient through that unit (dead ReLU).

Saturation sigmoid/tanh saturate at large $|z| \Rightarrow$ vanishing gradient; ReLU avoids positive saturation but can die.

Adjoint variables For computation graph nodes v_i , the adjoint is $\bar{v}_i := \partial L / \partial v_i$. Reverse-mode recurrence:

$$\bar{v}_i = \sum_{j: i \rightarrow j} \bar{v}_j \frac{\partial v_j}{\partial v_i}$$

Start with $\bar{L} = 1$ and send gradient messages backward. If a node has many children, adjoints add. Backprop = reverse-mode AD: one forward pass stores intermediates, one backward pass computes all parameter gradients.

Finite differences: needs $O(P)$ or $O(2P)$ loss calls for P params. **Backprop**: roughly one forward + one backward, linear in graph size.

Xavier/Glorot init $\text{Var}(W) \approx 2/(n_{\text{in}} + n_{\text{out}})$ for tanh/sigmoid.

He init $\text{Var}(W) \approx 2/n_{\text{in}}$ for ReLU.

Multi-Head Attention & Transformer Block

Multi-head attention h parallel heads with separate $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$ each projecting to $d_k = d_{\text{model}}/h$. Concatenate outputs and project by W_O . **Different heads attend to different relations/subspaces.**

Pre-norm decoder block

$$h' = h + \text{MHA}(\text{LN}(h)), \quad h_{\text{new}} = h' + \text{MLP}(\text{LN}(h'))$$

MLP usually $d \rightarrow 4d \rightarrow d$ (most parameters live here).

Sinusoidal positional encoding

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d}), \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d})$$

Absolute PE adds r_i to token i ; relative PE depends on distance $i - j$.

Rotation matrix properties

$$R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix}, \quad R(\alpha)^T = R(-\alpha) = R(\alpha)^{-1}, \quad R(\alpha)R(\beta) = R(\alpha + \beta), \quad \det R = 1$$

$R(\alpha)$ preserves length and dot products: $\|R(\alpha)x\| = \|x\|$, $(R(\alpha)x)^T R(\alpha)y = x^T y$.

RoPE shift-invariance

$$\text{RoPE}(x, m+k)^T \text{RoPE}(y, n+k) = x^T R((n-m)\theta)y$$

Same result regardless of global shift k . Use absolute PE for fixed context / simplicity; use RoPE / relative PE for length generalization (modern LLM default).

Why self-attention path length between any two tokens is $O(1)$ and parallelizable. RNN: $O(n)$, sequential. CNN: grows with depth/receptive field. **Cost: $O(n^2d)$ memory/time — all-pairs interactions.**

Autoregressive LMs & Decoding

Factorization $p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$, $L = -\sum_t \log p_\theta(x_t \mid x_{<t})$.

Teacher forcing train with ground-truth previous tokens; inference conditions on generated tokens (**exposure bias**).

Decoding strategies

- Greedy: $\arg \max$ logit (deterministic, repetitive).
- Temperature: $p_i \propto e^{z_i/\tau}$. $\tau \rightarrow 0$ greedy; $\tau \rightarrow \infty$ uniform.
- Top- k : keep k most likely tokens, renormalize.
- Top- p (nucleus): smallest set with cumulative mass $\geq p$.

Fine-Tuning: SFT, RLHF, DPO

SFT Supervised fine-tuning: cross-entropy on prompt/output pairs. Teaches formats and behaviors.

RLHF (1) train reward model from human preferences; (2) RL (e.g. PPO) against reward model with KL penalty to base.

DPO Directly optimize preference pairs (y^+, y^-) via classification-style loss. **No explicit reward model or RL loop — simpler, more stable.**

LoRA $W' = W + AB$, $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times d}$, $r \ll d$. Freeze W , train only A, B ($\sim 10000\times$ fewer params).

Synthetic data can teach formats, behaviors, domain knowledge.

Contrastive Learning Details (SimCLR / CLIP / temperature)

InfoNCE with temperature τ

$$\mathcal{L}_i = -\log \frac{e^{s_+/ \tau}}{\sum_j e^{s_j / \tau}}, \quad s_j = \text{sim}(z_i, z_j)$$

Small τ : emphasizes hard negatives (sharper). Large τ : smooths distribution.

SimCLR positives = two augmented views of same image; negatives = other images in batch.

CLIP positives = matched image-text pairs; negatives = mismatched pairs in batch. Contrastive features often outperform reconstruction features for semantic/discriminative downstream tasks.

Agents: RAG, ReAct, Tool Use

LLM maps tokens to tokens (no internal world state).

RAG Retrieval-Augmented Generation: fetch relevant text from a corpus and prepend to prompt. Reduces hallucination, enables fresh knowledge.

Tool use model emits structured calls (function name + args); host executes and returns results.

Agent loop (ReAct) Reason $\rightarrow$ Act $\rightarrow$ Observe $\rightarrow$ repeat until goal. State accumulates as conversation history.

Eval differences LM evals: single-turn Q/A. Agent evals: environments, trajectories, actions, final success. Long-horizon agents fail from compounding errors over many steps.

Improvement loops (a) SFT on successful trajectories; (b) RL on environment rewards; (c) GEPA-style prompt evolution using trajectory + reward reflection.

Protein AI

Sequence $\rightarrow$ structure amino acid sequence largely determines 3D fold and function. Tasks: structure prediction, function prediction, design, interaction prediction.

MSA / co-evolution multiple sequence alignment; correlated mutations across homologous proteins suggest 3D contact (residues that mutate together are likely close in structure). Foundation of AlphaFold-style methods.

Design strategies

- Physics-based: Rosetta, energy minimization.
- Directed evolution: iterate mutate $\rightarrow$ select.
- ML generative: $p_{\theta}(x)$ over sequences.
- ML predictive: train $p(y | x)$ then optimize.

Conditional design want $x \sim p(x | y) \propto p(y | x)p(x)$. Three approaches: (1) train $p_{\theta}(x | y)$ directly; (2) update unconditional $p(x)$ using property data (CbAS / preference-style); (3) guide a frozen generative model at generation time using $\nabla_x \log p(y | x)$.

Pitfall predictor maximization can exploit OOD errors $\Rightarrow$ non-folding / pathological sequences. Mitigations: priors, uncertainty estimates, hard constraints, wetlab validation.

Epistasis effect of mutations is non-additive / context-dependent — mutation A's effect depends on whether B is also present. Makes design hard; pure additive models miss this.

PyTorch

Train step

```
model.train()
for x, y in loader:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad()          # clear old grads
    pred = model(x)
    loss = criterion(pred, y)
    loss.backward()                 # compute new grads
    optimizer.step()                # apply them
```

Eval step

```
model.eval()                        # dropout off, BN uses running stats
with torch.no_grad():               # no autograd graph -> saves memory
    for x, y in loader:
        pred = model(x.to(device))
        loss = criterion(pred, y.to(device))
```

Learnable parameters `self.bias = nn.Parameter(torch.zeros(d)).`

Conv1d shape-preserving padding odd k : `padding=(k-1)//2`. Output: $\lfloor (N + 2P - F)/S \rfloor + 1$.

Dropout (inverted, train-time scaling)

```
if self.training:
    mask = (torch.rand_like(x) > self.p).float()
    return x * mask / (1 - self.p)
return x
```

LayerNorm core

```
mean = x.mean(dim=-1, keepdim=True)
std = x.std(dim=-1, keepdim=True)
return self.weight * (x - mean) / (std + 1e-5) + self.bias
```

Pre-norm residual block

```
x = x + conv(norm1(x))
x = x + mlp(norm2(x))
```

Sequential needs Modules, not functions

```
nn.Sequential(nn.Linear(d, d), nn.ReLU(), nn.Linear(d, d))
```

Use `nn.ReLU()` (module), not `F.relu` (function) – `Sequential` only accepts `nn.Module` instances.

Lists of submodules: `nn.ModuleList`

```
self.heads = nn.ModuleList([nn.Linear(d, d) for _ in range(n_heads)])
```

Plain Python lists don't register params. Use `nn.ModuleList` (indexable, no `forward`) or `nn.Sequential` (chained).

train() vs eval() — what flips Dropout: train zeroes+rescales, eval identity. BatchNorm: train uses batch stats (updates running), eval uses running stats. LayerNorm/Linear/Conv: unchanged.

Param counts Conv2d: $C_{\text{in}}C_{\text{out}}k_hk_w + C_{\text{out}}$. Linear: $d_{\text{in}}d_{\text{out}} + d_{\text{out}}$. LN/BN: 2·feat. Pool/ReLU/Dropout/residual: 0. LoRA on $D \times D$ at rank r : $2Dr$.

PyTorch Essentials

Big concepts

- `torch.tensor`: basic datatype for data, weights, activations (similar to `numpy.ndarray`).
- Neural networks built from primitives in `torch.nn`; define a **forward** method for prediction.
- PyTorch uses **automatic differentiation** to efficiently compute gradients.
- Training = (you) implementing some variant of gradient descent.

torch.tensor Basic building block; similar to `numpy.ndarray`.

- **Construction:** `torch.tensor([[1.0], [2.0]]); torch.ones(2,3)`
- **Indexing/slicing:** `x[:, 3]`
- **Shape and type:** `x.shape; x.dtype`
- **Operations:** `torch.mean(x, axis=1); a.T @ x`

Key differences from NumPy:

- **Device placement:** `x.to("cuda"); x.to("cpu")`
- Tensors can track how they were computed to enable **automatic differentiation**: `x.grad_fn`

torch.nn.functional vs torch.nn

- **torch.nn.functional:** **stateless** NN functions (e.g., `relu`).
- **torch.nn:** **basic building blocks** of neural networks (carry state/parameters).
- **nn.Module:** base class all models and layers extend.
 - `__init__`: defines all model parameters.
 - `forward (__call__)`: defines model/layer logic.
- **nn.Parameter:** wraps parameter tensors so they're tracked automatically (used in `__init__`).
- Hundreds of layer modules available: `nn.Linear`, `nn.Softmax`, ...

Building a model by extending nn.Module

```
class MLPModel(nn.Module):
    def __init__(self, dims, activation="relu"):
        super().__init__()
        self.layers = nn.ModuleList(
            [nn.Linear(dims[i], dims[i+1]) for i in range(len(dims)-1)]
        )
        if activation == "relu": self.act = nn.functional.relu
        elif: self.act = torch.tanh

    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i < len(self.layers)-1:
                x = self.act(x)
        return x
```

Computing gradients with autograd PyTorch automatically and efficiently computes the gradient of the loss w.r.t. all parameters.

```
model.zero_grad()          # reset gradients (needed each iteration)

pred = model(x)
loss = loss_fn(pred, t)    # compute loss

loss.backward()            # compute gradient of loss
print(model.linear.weight.grad) # gradients attached to each parameter
```